

业务全景与端到端主线

gomall 一次购物的端到端全景——从进门到收货怎么跑通 / 卡在哪

gomall · 业务侧 deck

业务全景：gomall 到底在做什么

系统架构：分层拓扑与领域地图

端到端主线：一次成功购物的”黄金路径”

贯穿全局的硬骨头（精选 3 个）

订单状态机：把 7 跳串成一条可查的线

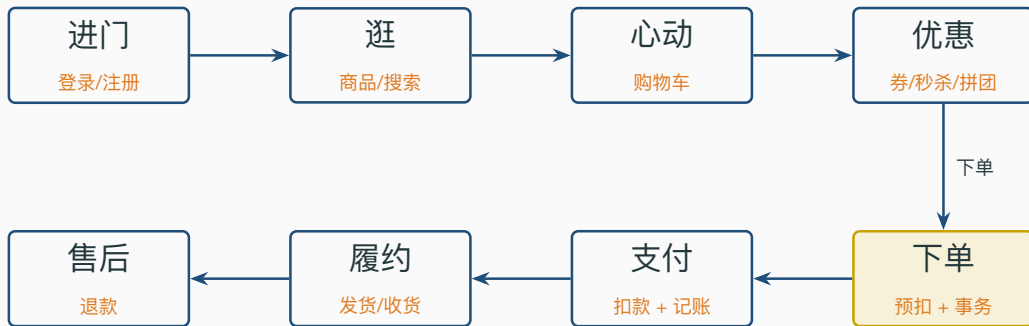
业务全景：gomall 到底在做什么

先认人：一笔订单背后站着 5 个角色

- **C 端用户**：进门 → 逛 → 下单 → 收货。只关心快、对、不丢。
- **商家 (Boss)**：订单是”我能不能发货”的唯一源头；货卖出去钱要到账。
- **运营**：办活动拉量——优惠券 / 满减 / 秒杀 / 拼团 / 预售，预算不能被刷穿。
- **客服**：用户来问”我的钱去哪了 / 货到哪了”，得有一条能查的状态线。
- **SRE**：大促洪峰别把库存卖超、别把数据库打挂，挂了要能降级。

这份概览把一次购物从**进门到收货**串成一条线：先看全局怎么连，再逐段看每一跳背后的技术取舍，以及”出事会卡在哪”。

业务全景图：一次购物穿过的 8 个环节



贯穿全局：库存防超卖·Outbox 一致性·流量治理·优雅降级

业务承诺：我们对外兜什么底（SLO）

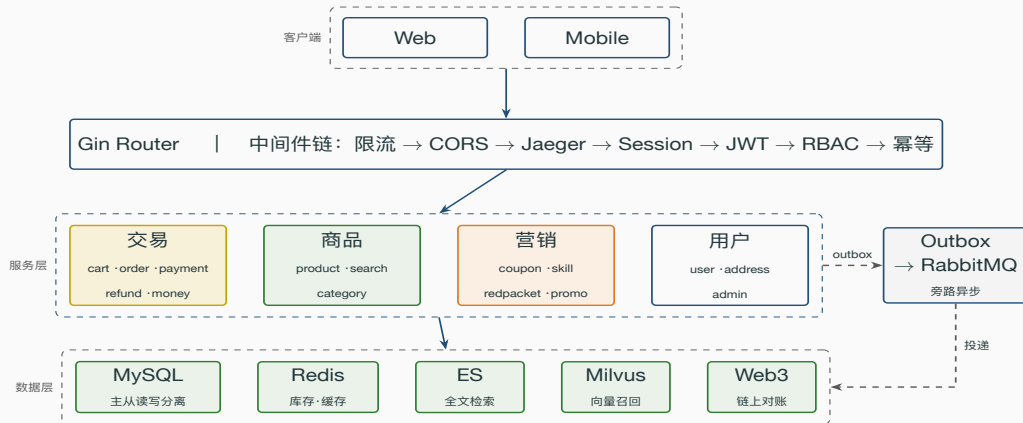
一个交易系统的可信度，不在于功能多，而在于敢把哪几条线写进合同——出了问题要赔的那种。gomall 把承诺分成 P0~P3 四档，资源向核心交易倾斜。

等级	典型场景	可用率	p99 延迟
P0 核心交易	下单 / 支付	99.95%	< 500ms
P1 核心读	商品详情 / 订单列表	99.9%	< 200ms
P2 营销秒杀	抢券 / 秒杀 / 红包	99%	< 200ms
P3 信息类	轮播 / 分类	99%	< 1s

- **钱货不丢**：支付与库存、资金流水落同一事务，崩了整笔回滚，绝不出现“扣了钱没货”。
- **happy path 不被周边阻断**：优惠 / 搜索 / 通知挂了，下单支付照走。
- **宁可下游慢、不能上游 429**：P0 接口（下单 / 支付）靠缓存 + 异步削峰扛峰值。

系统架构：分层拓扑与领域地图

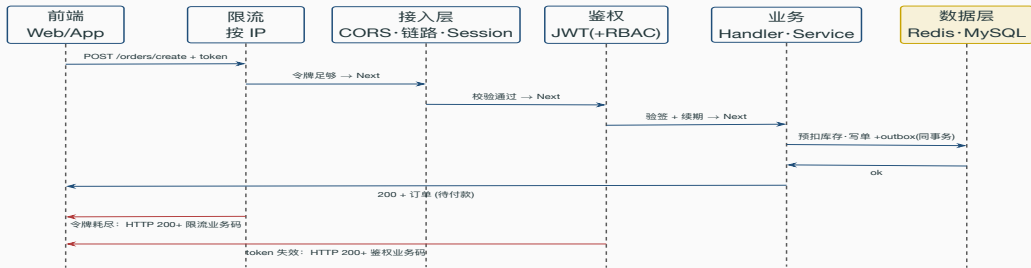
gomall 整体架构：一条请求从客户端落到数据层



核心链路同步落库收钱，事件经 Outbox 旁路异步散给搜索 / 统计 / 履约，主线不被下游拖慢。

中间件洋葱模型：下单请求逐层穿过鉴权打到后端（真实顺序）

中间件是洋葱模型：每层验过自己的关就调 `c.Next()` 递给内层，内层返回后再逐层收回；任一层 `Abort`，请求在那层原路返回，到不了业务。



真实顺序：限流 (IP) → CORS → Jaeger → Session → JWT；admin 再叠 RequireRole、下单末端挂 Idempotency。限流在最外层，洪水在最便宜处被拒；按用户的秒杀滑窗限流则在鉴权之后。 *routes/router.go:41-55* 全局链 + 分组；*internal/order/routes.go:12* 下单叠 Idempotency；被拒 6/14

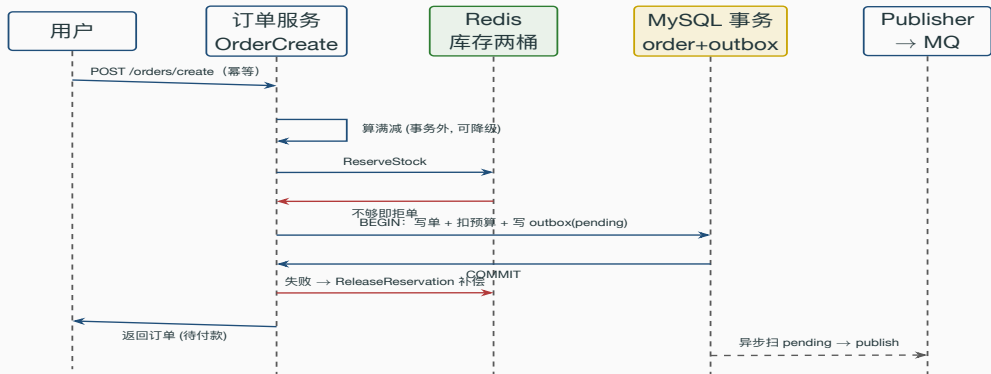
端到端主线：一次成功购物的”黄金路径”

黄金路径：7 跳，每一跳都可能掉链子

1. 登录——拿到 JWT，后续每个请求带着它过鉴权中间件。
2. 看商品——详情走 Cache Aside，热点不打穿 DB。
3. 加购 / 选地址——没地址下不了单，默认地址兜底。
4. 下单——算优惠 → 预扣库存 → 一个事务落订单 + 事件。
5. 支付——扣余额 + 改状态 + 真正消库存，同事务发”已支付”事件。
6. 发货 / 收货——状态机推进，每一步只允许合法迁移。
7. 结清——订单 Completed；中途不付钱则 15 分钟自动关单。

这条线横跨 user / product / cart / promo / order / inventory / payment 七个包，
任何一跳的失败都不能让用户”钱货两失”——这是整套设计的第一性原则。

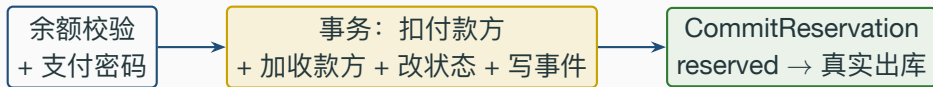
端到端时序：下单这一跳的内部（先预扣、再事务、后通知）



internal/order/service.go:110 Reserve :117 Tx :145 outbox :159 释放补偿:172 延迟关单

支付这一跳：钱和库存在这里才真正落定

- 下单时库存只是 **reserved** (预占)，钱还没动——订单状态 WaitPay。
- 支付成功才在一个事务里：扣用户余额 → 加商家余额 → 改状态 WaitShip → 写 order.paid 事件。
- 事务提交后同步 CommitReservation：把 reserved 真正消掉，库存数字才最终减少。



internal/payment/service.go:36 PayDown; repository/cache/inventory.go:98 CommitReservation

贯穿全局的硬骨头（精选 3 个）

硬骨头 1：库存防超卖——为什么不用 UPDATE WHERE stock>0

- 高并发下行锁排队 + 尾延迟爆炸；下单链路最怕在事务里等锁。
- gomall 把库存搬到 Redis，用 **两个桶**：available（可卖）/ reserved（已占）。
- 预扣 / 提交 / 释放都是 **单个 Lua 脚本原子完成**，一次 RTT，天然不超卖。

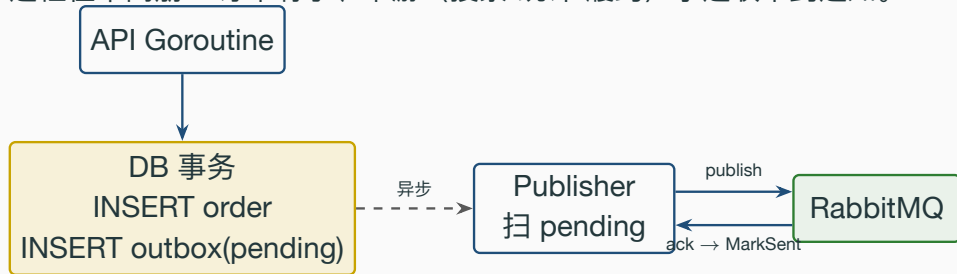
Listing 1: reserveScript: available 与 reserved 两个 key 原子流转

```
1 -- KEYS[1]=available 桶 KEYS[2]=reserved 桶 ARGV[1]=n
2 local avail = redis.call('GET', KEYS[1])
3 if avail == false then return -2 end -- 库存未初始化
4 if tonumber(avail) < tonumber(ARGV[1]) then
5     return -1 -- 不够，拒单
6 end
7 redis.call('DECRBY', KEYS[1], ARGV[1]) -- available -= n
8 redis.call('INCRBY', KEYS[2], ARGV[1]) -- reserved += n
9 return 1
```

repository/cache/inventory.go:32 reserveScript ·:78 ReserveStock 封装·:98/:113 Commit/Release

硬骨头 2: Outbox ——解决” 订单存了但消息丢了”

双写问题：先 INSERT order 再 publish MQ，两个动作不在一个事务里。
进程在中间崩 = 订单有了、下游（搜索/统计/履约）永远收不到通知。



订单与事件**同生共死**写进一个事务；投递交给独立 publisher 重试。
至少一次投递 + 下游幂等消费 = 事件不丢。

业务痛点：用户手抖狂点”提交订单”、客户端超时自动重试、弱网下 App 补发请求——同一笔下单不能变成三笔。

请求态状态机 (Redis Hash)

- init: 已发券、未执行
- processing: 首单执行中, 后到一律挡回
- done: 已完成, 缓存响应直接回放

一段 Lua 单次 RTT 原子完成”读态 + 抢锁”。

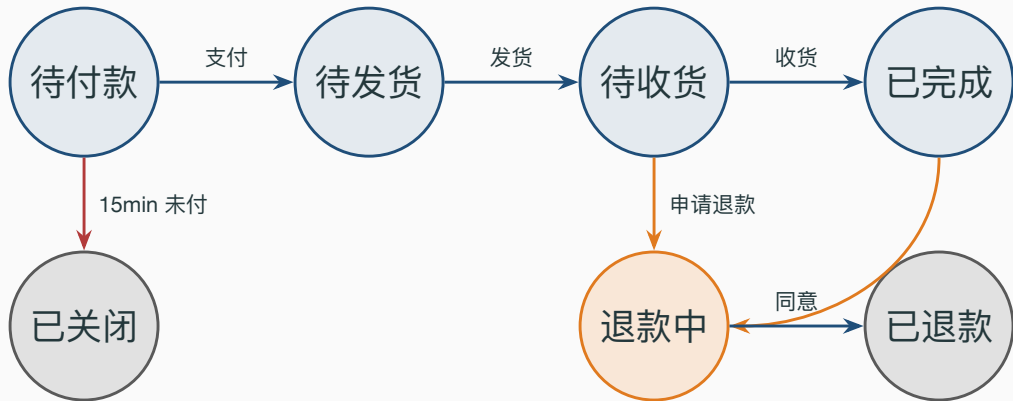
Listing 2: acquire: init 到 processing 原子推进

```
1 local v = redis.call('HGET', KEYS[1], 'state')
2 if v == false then return {0, ''} end
3 if v == 'done' then return {2,
4     redis.call('HGET', KEYS[1], 'result')} end
5 if v == 'processing' then return {3, ''} end
6 -- init: 抢到执行权
7 redis.call('HSET', KEYS[1], 'state', 'processing')
8 redis.call('EXPIRE', KEYS[1], tonumber(ARGV[1]))
9 return {1, ''}
```

压测实证：同 Idempotency-Key、50 VU、15s 累计 **755,033** 次请求 → DB 中

订单状态机：把 7 跳串成一条可查的
线

订单不是一个布尔值，是一台状态机



每一次状态变更都走 CanTransit 校验，非法迁移直接报错——
“已完成”的订单不可能跳回“待付款”，客服查到的状态永远自治。

这套架构的一句话总结

核心链路（扣库存 + 落单 + 收钱）必须稳如磐石，周边能力（优惠 / 通知 / 搜索 / 关单）全部可降级。

顺序即设计：

- 能在事务外做的绝不进事务——满减、库存预扣。
- 进了事务就保证原子——订单 + 优惠预算 + outbox 事件。
- 事务外的副作用必须能补偿——库存释放、延迟关单。

internal/order/service.go:63-178 OrderCreate —函数浓缩全部原则